Ad Soyad: _____	Tarih: _____
Lab 11 – Hafta 11 – *Eşzamanlılık Desenleri: Thread-Safe Yapılar, Paralel I/O, İlerleme/İptal, OBYS Responsive*	

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 11'de ele alınan eşzamanlılık desenlerini uygulamalı deneyeceksiniz. Önce bir thread-safe queue tasarlayacak, sonra sharded ConcurrentMap inşa edecek, ardından paralel CSV içe aktarımı, en son ise OBYS'nin mini responsive sürümünü ayağa kaldıracaksınız.
Süre: 3 saat **Gereksinimler:** g++ (C++20 desteği), `-pthread` bayrağı, Qt 6 (son çalışma için)

Derleme Komutları

Saf C++ örnekleri için:

```
g++ -std=c++20 -pthread -O2 -o program program.cpp
./program
```

Qt örnekleri için CMake kullanacağız (son çalışma).
Lab başlangıç kodları: `out/LabExamples/Lab11_Esamanlilik_Desenleri/` dizininde mevcut. Her çalışma için `calismaN.cpp` dosyasında `// GOREV` yorumlarını dolduracaksınız.

1 Çalışma 1: Thread-Safe Queue (45 dk)

Ön Bilgi

Bölüm 11 Ünite 1 Konu 1'de gördüğümüz `ThreadSafeQueue<T>` şablonunu sıfırdan yazacaksın. Hedef: tryPop, waitAndPop, kapat metodlarını yarış üretmeyecek şekilde tasarlamak.

Adım 1: İskelet Kodu İncele

Listing 1: ThreadSafeQueue İskeleti

```
template <typename T>
class ThreadSafeQueue {
    mutable std::mutex m_;
    std::condition_variable cv_;
    std::queue<T> kuyruk_;
    bool kapali_ = false;
public:
    void push(T deger);
    std::optional<T> tryPop();
    bool waitAndPop(T& cikti);
    void kapat();
    bool kapaliMi() const;
};
```

Adım 2: Görevler

1. **push**: Mutex korumalı kuyruğa ekleme yap, *kilit serbest kaldıktan sonra* notify_one çağır.
2. **tryPop**: Boşsa `std::nullopt`, doluysa move ile çıkar.
3. **waitAndPop**: Predicate ile wait; ya iş geldi ya kapali. Boşsa `false`.

4. **kapat**: Bayrağı `true` yap, notify_all.
5. Test: 1 üretici 100 sayı atar, 3 tüketici sırayla çeker.

notify_one'i *kilit altında* çağırırsan uyandırılan thread hemen mutex'e takılıp tekrar bekler – gereksiz bağlam değiştirme. Daima `{ ...}` bloğunun dışında çağır.

Beklenen Çıktı

```
Tuketici 1: 1
Tuketici 2: 2
Tuketici 3: 3
... (toplam 100 satır, dağılım rastgele)
Kuyruk bos. Kapali: 1
```

2 Çalışma 2: Sharded ConcurrentMap (45 dk)

Ön Bilgi

Tek mutex'li sözlük 8 thread paralel okuduğunda darboğaz olur. Sharding (bölmeleme) ile her bölmeye ayrı mutex verip paralelliği N kat artırırız.

Adım 1: Bölme Yapısı

Listing 2: Bölme Tasarımı

```
struct Bolme {
    mutable std::mutex m;
    std::unordered_map<int, std::string> veri;
};

class ConcurrentMap {
    std::deque<Bolme> bolmeler_;      // <-- vector DEGIL!
public:
    explicit ConcurrentMap(std::size_t n) : bolmeler_(n) {}
    void koy(int k, std::string v);
    std::optional<std::string> al(int k) const;
    bool sil(int k);
    std::size_t boyut() const;
};
```

Adım 2: Görevler

1. **bolmeBul**: `hash(k) % N` ile bölmeyi seç.
2. **koy/al/sil**: Bölmenin mutex'ini kilitle, iç unordered_map üzerinde işlem yap.
3. **boyut**: Tüm bölmeleri sırayla kilitle, toplam ver. Snapshot değil!
4. Test: 8 thread her biri 5000 anahtar yazar, sonra 8 thread paralel okur. Hata olmamalı.

[Vector vs Deque] `std::vector<Bolme>` büyürken iç dizisini taşımaya çalışır; `std::mutex` taşınamadığı için **derleme hatası**. Çözüm: `std::deque` veya `std::vector<std::unique_ptr<Bolme>>`.

3 Çalışma 3: Paralel CSV + jthread + stop_token (60 dk)

Ön Bilgi

50.000 satırlık bir CSV'yi tek thread'le okumak yavaştır ve UI'yı dondurur. Üretici-tüketici boru hattıyla 1 okuyucu + N parser kullanarak hem hızlandırır hem iptal edilebilir hale getiririz.

Adım 1: Test CSV'sini Oluştur

Listing 3: Test verisi üretimi

```
void testVerisiOlustur(const std::string& yol, int n) {
    std::ofstream f(yol);
    f << "numara,ad,bolum,gpa\n";
    for (int i = 1; i <= n; ++i) {
        f << (100000 + i) << ",Ogrenci_" << i
          << ",Bilgisayar," << (2.0 + (i % 200) * 0.01) << '\n';
    }
}
```

Adım 2: Görevler

1. **ThreadSafeQueue<std::string>**: Çalışma 1'den kullan.
2. **Üretici**: ifstream ile satır satır oku, push et.
3. **Parser işçileri**: waitAndPop ile satır al, parse et, paylaşılan haritaya ekle (shared_mutex kullan).
4. **İptal**: jthread + stop_token kullan; ana thread 500 ms sonra request_stop çağırırsa işçiler durmalı.
5. **İlerleme**: Atomic sayaçla her saniye kaç satır işlendiğini yazdır.

Beklenen Davranış

İptalsiz: 50.000 satır 200-400 ms'de yüklenir.
İptal: "Iptal istegi geldi (i=∼10000)" satırı görünür, programın geri kalan satırı işlenmez.

4 Çalışma 4: Atomic Sayaç + ASCII Progress Bar (30 dk)

Ön Bilgi

4 worker'ın paralel iş yaptığı bir senaryoda ortak bir std::atomic<size_t> sayacı toplam ilerlemeyi tutar. Ana thread her 100 ms'de okuyup ASCII progress bar çizer.

Görevler

1. 4 jthread, her biri 1000 iş yapar (her iş 1-5 ms uyku).
2. Atomic sayaca fetch_add (memory_order_relaxed).
3. Ana thread sayaç < 4000 olduğu sürece progress bar çiz, 100 ms uyu.
4. Bar genişliği 40 karakter; doluluk yüzdesine göre # ve . karakterler.
5. Sonunda toplam süre yazdır.

Beklenen Çıktı

```
[#####################################....] 90.0% (3600/4000)
[#######################################] 100.0% (4000/4000)
Toplam sure: 1500-3500 ms
```

5 Çalışma 5: Mini OBYS Responsive (40 dk)

Ön Bilgi

Bu son çalışmada Ünite 4'teki mini projenin küçük bir sürümünü kuracaksın. QProgressDialog + Worker pattern ile içe aktarma. Tam proje başlangıç kodları `calisma5/` altında.

Görevler

1. QThread oluştur, worker'ı moveToThread ile taşı.
2. QProgressDialog kur (modal, minimumDuration=500).
3. Sinyal/slot bağlantıları:
   - Worker::ilerleme → Dialog::setValue
   - Dialog::canceled → Worker::durdur (DirectConnection)
   - Worker::tamamlandi → MainWindow::raporGoster + temizlik
4. Test: 10.000 satırlık CSV import. UI'nin canlı kaldığını test et: dialog açıkken ana pencereyi taşımayı dene.

Beklenen Davranış

- Dialog 500 ms sonra açılır.
- İlerleme çubuğu 4-5 saniyede dolar.
- Bu süre boyunca pencere taşınabilir, menüler tıklanabilir.
- İptal'e basılırsa 1 saniye içinde kapanır; yarım yüklenen kayıtlar depoda kalır.

Değerlendirme

Çalışma	Puan
Çalışma 1 – ThreadSafeQueue	20
Çalışma 2 – Sharded ConcurrentMap	20
Çalışma 3 – Paralel CSV + jthread	25
Çalışma 4 – Atomic + Progress Bar	15
Çalışma 5 – Mini OBYS Responsive	20
Toplam	**100**

Yarış Tespiti

Lab tamamlandıktan sonra ThreadSanitizer ile programı tekrar derleyin:

```
g++ -std=c++20 -O1 -g -fsanitize=thread -pthread \
    -o calisma1_tsan calisma1.cpp
./calisma1_tsan
```

Eğer mutex'siz paylaşım yaptığınız bir yer varsa TSan size satır numarasıyla raporlar. Tüm çalışmalarınız TSan'i temiz geçmeli.